

Blood Bank & Blood Donation Management System

TAE-2 | DBMSProject

Student Details

Student: Shivam Yadav

Roll No.: P54

College: G.H. Rasoni College of Engineering and Management, Pune

Branch: B.Tech CSE – Data Science

Project Stack

Subject: DBMS

RDBMS: PostgreSQL

GUI: pgAdmin 4

Database: blood_bank

TAE-2 PROJECT: Domain & Normalisation

Domain: Blood Bank Management

Domain covers donor registration, donation management, blood-unit tracking, screening, component preparation, inventory & storage, hospital requests, and blood issue.

Normalisation Path



UNF

1NF

2NF

3NF

BCNF

12 entities | REQUEST_DETAIL resolves M:N relationship

12 Final Relations

DONOR

BLOOD_GROUP

DONATION

BLOOD_UNIT

BLOOD_SCREENING

BLOOD_COMPONENT

STORAGE_LOCATION

BLOOD_INVENTORY

HOSPITAL

BLOOD_REQUEST

REQUEST_DETAIL

BLOOD_ISSUE

System Architecture & Implementation Stack

Architecture: 6-Step Process

01

Raw Dataset

1,250 source records as input

02

SQL Transformation & Population

Data cleansing
and loading via DML

03

PostgreSQL Database

blood_bank schema instantiated

04

12 Relational Table

Fully normalised to BCNF

05

Views + Stored Procedure + Trigger + indexes

Business logic and performance layer

06

SQL Query Execution & Reporting

Advanced SQL analysis and output

Implementation Stack

RDBMS: PostgreSQL 18

GUI: pgAdmin 4

Database: blood_bank

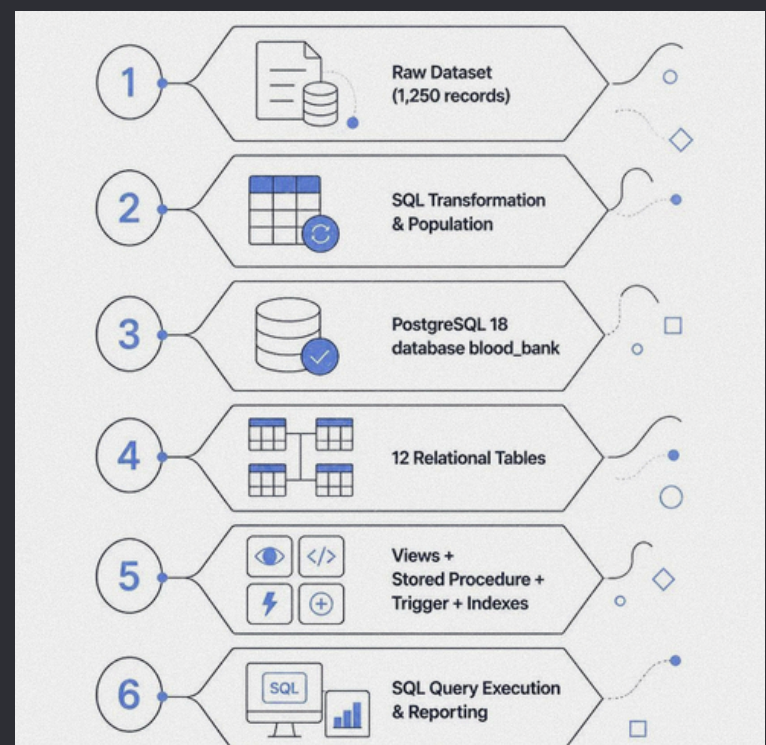
SQL: DDL + DML + Advanced SQL

SQL Layers Used

DDL
Schema creation

DML
Data population

Advanced SQL
Joins, subqueries, aggregation



Database Schema & Entity Relationships

12 interconnected entities with key relationships spanning the full blood donation lifecycle:

Core Donation Chain

- **DONOR (1) → DONATION (N)**
Each donor may have multiple donation records
- **BLOOD_GROUP (1) → DONATION (N)**
Blood group classifies each donation
- **DONATION (1) → BLOOD_UNIT (N)**
Each donation yields one or more blood units
- **BLOOD_UNIT (1) → BLOOD_SCREENING (N)**
Units undergo screening before use
- **BLOOD_UNIT (1) → BLOOD_COMPONENT (N)**
Units separated into components
- **BLOOD_COMPONENT (1) → BLOOD_INVENTORY (1)**
Each component tracked in inventory

Request & Issue Chain

- **BLOOD_INVENTORY (1) → STORAGE_LOCATION (N)**
Inventory mapped to storage locations
- **HOSPITAL (1) → BLOOD_REQUEST (N)**
Hospitals raise multiple requests
- **BLOOD_REQUEST (1) → BLOOD_ISSUE (N)**
Requests fulfilled via issue records
- **BLOOD_REQUEST (M) ↔ REQUEST_DETAIL (N)**
Junction table resolves M:N relationship
- **REQUEST_DETAIL (N) → BLOOD_COMPONENT (1)**
Detail specifies component requested
- **REQUEST_DETAIL (N) → BLOOD_INVENTORY (1)**
Detail links to inventory record

Integrity Constraints & Schema Features

Integrity Constraints

NOT NULL

Prevents missing mandatory values across all critical fields

UNIQUE

Prevents duplicate values in key identifier columns

CHECK

Restricts invalid values — e.g. blood group codes, status enumerations

DEFAULT

provides predefined values where applicable to reduce null entries

PRIMARY KEY

Uniquely identifies each record in every table

FOREIGN KEY

Maintains referential integrity across all 12 relations

Schema Features

- 12 relational tables with Primary & Foreign Keys
- NOT NULL, UNIQUE, CHECK constraints applied throughout
- Default values defined for status and date fields
- ON DELETE / ON UPDATE referential actions configured

Special Constraint



Composite Primary Key:

REQUEST_DETAIL(Request_ID, Component_ID) — resolves the M:N relationship between BLOOD_REQUEST and BLOOD_COMPONENT

Data Population & Advanced SQL Implementation

665

Registered donors

1,250

Donation records

1,250

Units tracked

1,591

BLOOD_COMPONENT

Components prepared

1,591

BLOOD_INVENTORY

Inventory entries

1,250

Hospital requests

Additional tables: BLOOD_SCREENING: 1,250 | REQUEST_DETAIL: 1,250 | BLOOD_ISSUE: 1,250

Advanced SQL Implementation

Multi-Table Joins

- **INNER JOIN:** Donor + Donation + Blood Group
- **LEFT JOIN:** All donors + donation records
- **SELF JOIN:** Donors with matching last names

Aggregation

- **GROUP BY + HAVING** for filtered aggregation
- Blood-group-wise donation analysis

Correlated Subquery

- Identifies donors with more than 2 donations
- Row-level correlation with outer query

Stored Procedure

Stored Procedure: `issue_blood`

01

Validate Quantity

Checks requested quantity is valid before proceeding

02

Check Inventory

Locks inventory row with FOR UPDATE to prevent race conditions

03

Validate Request & Component

Confirms request and component records exist and are eligible

04

Insert Blood Issue Record

Creates a new BLOOD_ISSUE entry for traceability

05

Update Inventory

Reduces available quantity and increases fulfilled quantity atomically

Trigger: `trg_update_request_status`

Fires After Update

Activates after `Fulfilled_Quantity` is updated on `REQUEST_DETAIL`

Checks Pending Quantity

Compares fulfilled vs. requested quantity to determine status

Updates Status

Sets status to **Partially Fulfilled** or **Fulfilled** automatically

Preserves Terminal States

Does not overwrite **Cancelled** or **Rejected** status values

Views & Performance Optimisation

Database Views

vw_available_blood_inventory

- Displays available blood components
- Combines inventory and component information
- Simplifies availability checking

vw_hospital_request_summary

- Summarises hospital blood requests
- Shows request and fulfillment information
- Simplifies repeated reporting queries



Purpose: Provide reusable query results without repeatedly writing complex joins.

Performance Optimisation: 4-Step Process

01

Complex Query Written

Multi-join query targeting high-cost operations identified

02

EXPLAIN ANALYZE Run

Query cost analysis performed — baseline cost recorded

03

Composite Index Created

Targeted index built on high-selectivity column combinations

04

Re-run EXPLAIN ANALYZE

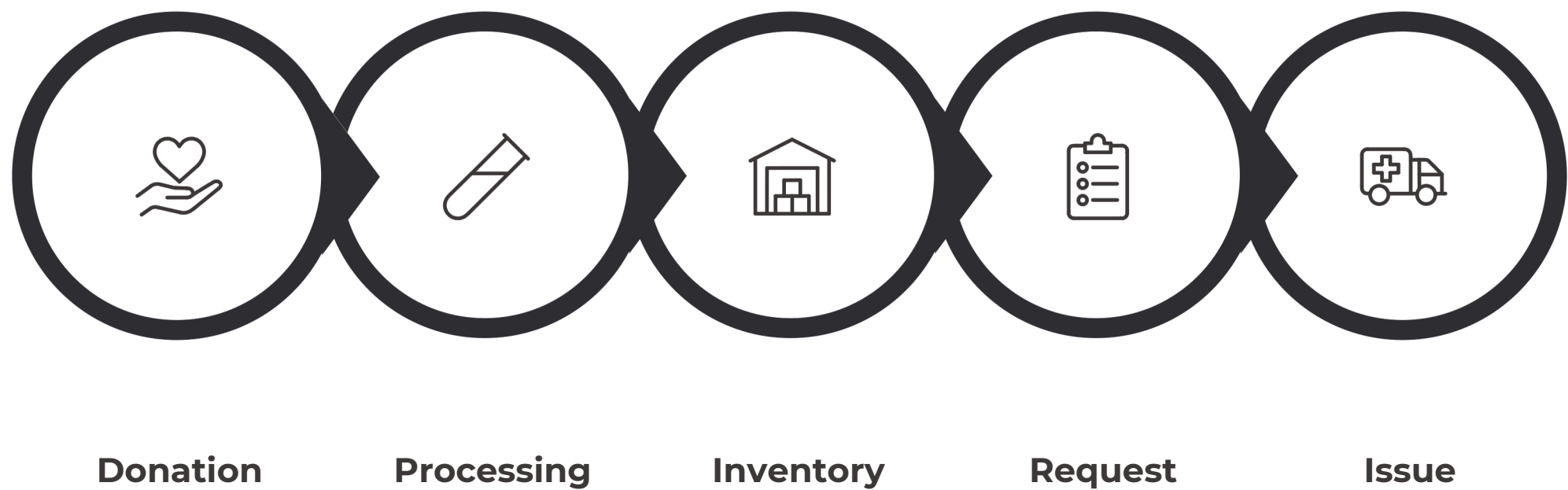
Post-index cost compared against baseline to confirm improvement

Indexes Created

- **Query 1:** `idx_donation_bloodgroup_date` (Blood_Group_ID, Donation_Date)
- **Query 2:** `idx_inventory_status_component` (Inventory_Status, Component_ID)

Integrated Blood Bank Workflow

Complete traceability from donation to hospital issue — every step linked through relational integrity:



Key Implementation Features

Constraints Data Integrity enforced at schema level across all 12 tables		Views Reporting simplified via two reusable business views		Stored Procedure Transaction processing for blood issue with row-level locking
Trigger Automated business rule for request-status lifecycle management		 Indexes Query optimisation via composite indexes with EXPLAIN ANALYZE validation		

Conclusion

TheBloodBank&BloodDonationManagement System demonstrates a complete, production-grade DBMS implementation covering schema design, data integrity, business logic automation, and query performance optimisation.

1 Schema & Data Population

Created 12 interconnected tables with integrity constraints; populated **1,250+ records** across all entities including 1,591 blood component and inventory entries.

3 Views & Stored Procedure

Created two business-reporting views (`vw_available_blood_inventory`, `vw_hospital_request_summary`); implemented stored procedure `issue_blood` for atomic blood issue transactions.

2 Advanced SQL

Implemented advanced joins (INNER, LEFT, SELF), correlated subqueries, and **GROUP BY / HAVING** analysis for blood-group-wise donation reporting.

4 Trigger & Performance Optimisation

Implemented trigger `trg_update_request_status` for request-status automation; applied **EXPLAIN ANALYZE** and composite indexes (`idx_donation_bloodgroup_date`, `idx_inventory_status_component`) for measurable query optimisation.